LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA

JOBSHEET 6:

SEARCHING

Aylafada Syakira

TI-1C

254107020116

PROGRAM STUDI D-IV TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG 2026

6.1. Tujuan Praktikum

Setelah melakukan materi praktikum ini, mahasiswa mampu:

1. Menjelaskan mengenai algoritma Searching.
2. Membuat dan mendeklarasikan struktur algoritma Searching.
3. Menerapkan dan mengimplementasikan algoritma Searching.

6.2. Searching/ Pencarian Menggunakan Algoritma Sequential Search

6.2.1. Langkah-langkah Percobaan Sequential Search

1. Membuat class Mahasiswa, MahasiswaBerprestasi, dan MahasiswaDemo pada

2. Menambahkan method sequentialSearching bertipe integer dengan parameter cari bertipe double pada class MahasiswaBerprestasi. Kemudian mendeklarasikan isi method sequentialSearching dengan algoritma pencarian data menggunakan teknik sequential searching.

```
int sequentialSearching (double cari) {
    int posisi = -1;
    for (int j=0; j<listMhs.length; j++) {
        if (listMhs[j].ipk==cari) {
            posisi = j;
            break;
        }
    }
    return posisi;
}
```

3. Membuat method tampilPoisisi bertipe void dan Deklarasikan isi dari method tampilPoisisi pada class MahasiswaBerprestasi06

```
void tampilPosisi (double x, int pos) {
    if (pos !=-1) {
        System.out.println("Data mahasiswa dengan IPK: " +x +"ditemukan pada indeks " +pos);
    } else {
        System.out.println("Data " +x +"tidak ditemukan" );
    }
}
```

4. Membuat method tampilDataSearch bertipe void dan mendeklarasikan isi dari method tampilDataSearch pada class MahasiswaBerprestasi.

```
void tampilDataSearch (double x, int pos) {
    if (pos !=-1) {
        System.out.println("nim\t: " +listMhs[pos].nim);
        System.out.println("nama\t: " +listMhs[pos].nama);
        System.out.println("kelas\t: " +listMhs[pos].kelas);
        System.out.println("ipk\t: " +listMhs[pos].x);
    } else {
        System.out.println("Data mahasiswa dengan IPK:  " +x +"tidak ditemukan ");
    }
}
```

5. Menambahkan kode program untuk melakukan pencarian data dengan algoritma sequential searching pada class MahasiswaDemo06

```java
public class mahasiswaDemo06 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);
        mahasiswaBerprestasi06 list = new mahasiswaBerprestasi06();
        int jumMhs=5;

        // System.out.print("Masukkan jumlah mahasiswa: ");
        // int jumlah = sc.nextInt();
        // sc.nextLine();

        for (int i=0; i<jumMhs; i++) {
            System.out.println("Data Mahasiswa ke-" +(i+1));

            System.out.print(s: "NIM: ");
            String nim = sc.nextLine();

            System.out.print(s: "Nama: ");
            String nama  = sc.nextLine();

            System.out.print(s: "Kelas: ");
            String kelas  = sc.nextLine();

            System.out.print(s: "IPK: ");
            double ipk  = sc.nextDouble();
            sc.nextLine();

            mahasiswa06 m = new mahasiswa06(nim, nama, kelas, ipk);
            list.tambah(m);
        }
        list.tampil();
        //melakukan pencarian data sequential
        System.out.println(x: "-----------------------------------------------");
        System.out.println(x: "Pencarian data");
        System.out.println(x: "-----------------------------------------------");
        System.out.println(x: "Masukkan ipk mahasiswa yang dicari: ");
        System.out.println(x: "IPK: ");
        double cari = sc.nextDouble();

        System.out.println(x: "Menggunakan sequential searching");
        double posisi = list.sequentialSearching(cari);
        int pss = (int)posisi;
        list.tampilPosisi(cari, pss);
        list.tampilDataSearch(cari, pss);
```

6.2.2. Verifikasi Hasil Percobaan

```
Data Mahasiswa ke-1
NIM: 111
Nama: ayla
Kelas: 1
IPK: 4
Data Mahasiswa ke-2
NIM: 222
Nama: gibran
Kelas: 2
IPK: 2,3
Data Mahasiswa ke-3
NIM: 333
Nama: wowo
Kelas: 3
IPK: 3
Data Mahasiswa ke-4
NIM: 444
Nama: joko
Kelas: 4
IPK: 2,5
Data Mahasiswa ke-5
NIM: 555
Nama: lia
Kelas: 2
IPK: 3,5
Nama: ayla
NIM: 111
Kelas: 1
IPK: 4.0
---------------------------------
Nama: gibran
NIM: 222
Kelas: 2
IPK: 2.3
---------------------------------
Nama: wowo
NIM: 333
Kelas: 3
IPK: 3.0
---------------------------------
Nama: joko
NIM: 444
Kelas: 4
IPK: 2.5
---------------------------------
Nama: lia
NIM: 555
Kelas: 2
IPK: 3.5
---------------------------------
-------------------------------------------------
Pencarian data
-------------------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK:
4
Menggunakan sequential searching
Data mahasiswa dengan IPK: 4.0ditemukan pada indeks 0
nim     : 111
nama    : ayla
kelas   : 1
ipk     : 4.0
```

6.2.3. Pertanyaan

1. Jelaskan perbedaan metod tampilDataSearch dan tampilPosisi pada class MahasiswaBerprestasi!
   - tampilPosisi berfungsi untuk menampilkan index data mahasiswa yang ditemukan berdasarkan IPK yang dicari dan memberikan informasi apakah data yang dicari ditemukan atau tidak. Sedangkan tampilDataSearch berfungsi untuk menampilkan detail data mahasiswa yang ditemukan seperti NIM, nama, kelas, dan IPK

2. Jelaskan fungsi break pada kode program di bawah ini!

```
if (listMhs[j].ipk==cari){
    posisi=j;
    break;
}
```

   - Menghentikan perulangan for segera setelah data ditemukan
   - Membuat proses pencarian efisien karena tidak perlu memeriksa data selanjutnya

3. Apa fungsi variabel pos atau indeks hasil pencarian dalam program sequential search?
   - Menujukkan Lokasi data dalam array listMhs
   - Menentukan aoakah data ditemukan atau tidak. Jika pos =1 maka data tidak ditemukan. Jika pos >= 0 maka data ditemukan
   - Digunakan untuk mengakses data mahasiswa, seperti pada listMhs[pos].nama;
   - Menjadi parameter bagi method tampilPosisi dan tampilDataSearch

4. Jika terdapat lebih dari satu data dengan nilai yang sama, hasil pencarian sequential search yang dibuat di atas akan menampilkan data ke berapa? Jelaskan
   - Akan menampilkan data pertama yang ditemukan

5. Berkaitan dengan pertanyaan nomor 2 di atas, apa yang terjadi jika perintah break dihapus dari kode di atas?
   - Perulangan akan terus berjalan hinga akhir
   - Nilai posisi akan terus diperbarui setiap kali menemukan kecocokan
   - Indeks yang dikembalikan adalah kemunculan terakhir, bukan yang pertama

6.3. Searching/ Pencarian Menggunakan Algoritma Binary Search

6.3.1. Langkah-langkah Percobaan Binary Search

1. Menambahkan method findBinarySearch bertipe integer pada class MahasiswaBerprestasi. Kemudian mendeklarasikan isi method findBinarySearch dengan algoritma pencarian data menggunakan teknik binary searching pada percobaan 6.2.1 (sequential search)

```
int findBinarySearch (double cari, int left, int right) {
    int mid;
    if (right>=left){
        mid = (left+right)/2;
        if (cari == listMhs[mid].ipk) {
            return mid;
        } else if (listMhs[mid].ipk>cari) {
            return findBinarySearch(cari, left, mid-1);
        } else {
            return findBinarySearch(cari, mid+1, right);
        }
    }
    return -1;
}
```

2. Memanggil method findBinarySearch terdapat pada class MahasiswaBerprestasi di kelas MahasiswaDemo. Kemudia memanggil method tampilPosisi dan tampilDataSearch

```
//melakukan pencarian data binary
System.out.println(x: "-----------------------------------------")
System.out.println(x: "Pencarian data ");
System.out.println(x: "-----------------------------------------")
System.out.println(x: "Masukkan ipk mahasiswa yang dicari: ");
System.out.print(s: "IPK: ");
cari = sc.nextDouble();
System.out.println(x: "-----------------------------------------")
System.out.println(x: "Menggunakan binary search");
System.out.println(x: "-----------------------------------------")
double posisi2 = list.findBinarySearch(cari, left: 0, jumMhs-1);
int pss2 = (int) posisi2;
list.tampilPosisi(cari, pss2);
list.tampilDataSearch(cari, pss2);
```

6.3.2. Verifikasi Hasil Percobaan

```
Data Mahasiswa ke-1
NIM: 111
Nama: lala
Kelas: 1
IPK: 4
Data Mahasiswa ke-2
NIM: 222
Nama: lili
Kelas: 2
IPK: 3
Data Mahasiswa ke-3
NIM: 333
Nama: lulu
Kelas: 4
IPK: 3,8
Data Mahasiswa ke-4
NIM: 444
Nama: lolo
Kelas: 1
IPK: 3,8
Data Mahasiswa ke-5
NIM: 555
Nama: lele
Kelas: 2
IPK: 4
Nama: lala
NIM: 111
Kelas: 1
IPK: 4.0
```

```
---------------------------------
Nama: lili
NIM: 222
Kelas: 2
IPK: 3.0
---------------------------------
Nama: lulu
NIM: 333
Kelas: 4
IPK: 3.8
---------------------------------
Nama: lolo
NIM: 444
Kelas: 1
IPK: 3.8
---------------------------------
Nama: lele
NIM: 555
Kelas: 2
IPK: 4.0
---------------------------------
-----------------------------------------
Pencarian data
-----------------------------------------
Masukkan ipk mahasiswa yang dicari:
IPK:
3,8
-----------------------------------------
Menggunakan binary search
-----------------------------------------
Data mahasiswa dengan IPK: 3.8ditemukan pada indeks 2
nim     : 333
nama    : lulu
kelas   : 4
ipk     : 3.8
```

6.3.3. Pertanyaan

1. Tunjukkan pada kode program yang mana proses divide dijalankan!
   - Pemanggilan rekursif ke kiri (left sampai mid -1) atau ke kanan (mid+1 sampai right) merupakan proses devide pada program. Ditunjukkan dengan kode program dibawah ini:

```
mid = (left+right)/2;
```

2. Tunjukkan pada kode program yang mana proses conquer dijalankan!
   - Tahap conquer terjadi Ketika Solusi ditemukan atau Ketika rekursi dihentikan. Ketika nilai yang dicari sama dengan listMhs[mid].ipk, maka indeks mid dikembalikan sebagai Solusi. Jika right<left, berati pencarian selesai dan data tidak ditemukan sehinga mengembalikan -1

```
if (cari == listMhs[mid].ipk) {
                return mid;
```

3. Apa fungsi left, right, dan mid?
   - Left digunakan untuk menunjukkan batas kiri (awal) dari area pencarian dalam array. Right digunakan untuk menunjukkan batas kanan (akhir) dari area

pencarian dalam array. Mid menunjukkan index Tengah yang digunakan untuk membandingkan nilai yang dicari dengan elemen array

4. Jika data IPK yang dimasukkan tidak urut. Apakah program masih dapat berjalan? Mengapa demikian?
   - Program tetap dapat dijalankan, tetapi hasil binary searchnya tidak akurat atau bisa salah. Karena binary search mengharuskan data dalam keadaan terurut ascending maupun descending. Jika data tidak terurut, pembagian ruang pencarian menjadi tidak valid sehingga lemen yang dicari bisa terlewat. Jadi sebelum melakukan binary search data harus mengurutkan data terlebih dahulu, misalnya dengan bubble sort atau yang lainnya

5. Jika IPK yang dimasukkan dari IPK terbesar ke terkecil (misal: 3.8, 3.7, 3.5, 3.4, 3.2) dan elemen yang dicari adalah 3.2. Bagaimana hasil dari binary search? Apakah sesuai? Jika tidak sesuai maka ubahlah kode program binary seach agar hasilnya sesuai
   - Tidak sesuai, karena kode binary search yang ada mengasumsikan bahwa data diurutkan dari terkecil ke terbesar atau ascending. Agar hasilnya sesuai, ubah kondisi if (listMhs[mid].ipk > cari) menjadi if (listMhs[mid].ipk < cari) agar sesuai dengan urutan descending.

6. Jelaskan bagaimana binary search menentukan bahwa data yang dicari tidak ditemukan di dalam array.
   - Ketika batas kiri melebihi batas kanan, Ketika seluruh pencarian telah diperiksa dan tidak ada kecocokan, kondisi left>right menandakan bahwa elemen tersebut tidak ada di dalam array. Nilai -1 digunakan sebagai indicator bahwa data tidak ditemukan

7. Modifikasi program di atas yang mana jumlah mahasiswa yang diinputkan sesuai dengan masukan dari keyboard.
   Tambahan program dari class mahasiswaDemo06:

```
Scanner sc = new Scanner (System.in);

System.out.print(s: "Masukkan jumlah mahasiswa: ");
int jumlah = sc.nextInt();
sc.nextLine();

mahasiswaBerprestasi06 list = new mahasiswaBerprestasi06(jumlah);
// int jumMhs=5;
```